

Technical Specification: Hybrid Export Engine (HTML & Blade)

1. Core Export Architecture

The export process relies on a **Two-Pass Serialization System**.

Instead of trying to write HTML manually, we will leverage React's built-in server-side rendering capabilities in the browser.

1. **Pass 1 (Stringification):** We feed the JSON state into a specialized set of React components (different from the editor components, optimized purely for static viewing). We use `ReactDOMServer.renderToStaticMarkup()` to instantly generate an HTML string.
2. **Pass 2 (Mutation & Packaging):** We take that raw HTML string, inject necessary CSS into the `<head>`, inline Base64 images, and (if exporting to Blade) run a RegEx engine to convert text placeholders into backend logic.

2. Resolving the "Expanding Table" Dilemma

In a hybrid system, if a user places a "Signature" 800px from the top of the page, and the dynamic table expands from 3 items to 30 items, the table will print right over the signature.

To fix this, the exporter uses an **Anchoring System**. Free-form elements in the JSON will have an `anchorMode`:

- `anchorMode: "page"` (Default): The element is absolutely positioned relative to the top-left of the entire document (e.g., a background watermark).
- `anchorMode: "block"`: The element is positioned relative to a specific Flow Block (e.g., anchored to the Totals Block). If the dynamic table pushes the Totals Block down, the floating element moves down with it perfectly.

3. HTML Export Strategy (The Single-File Output)

For pure HTML exports, the goal is **Zero External Dependencies**. The output must be a single .html file that a user can double-click, and it will render perfectly even offline.

Structure of the Exported HTML:

HTML

`<!DOCTYPE html>`

```

<html>
<head>
  <meta charset="UTF-8">
  <style>
    /* 1. Global Reset & Print Media Queries */
    @media print { body { -webkit-print-color-adjust: exact; print-color-adjust: exact; } }
    /* 2. Custom Font Injections (Base64 or Google Fonts) */
    /* 3. Base Document Styling */
    .document-wrapper { position: relative; width: 210mm; min-height: 297mm; background: white;
margin: 0 auto; overflow: hidden; }
  </style>
</head>
<body>
  <div class="document-wrapper">

    <div style="position: absolute; top: 10px; left: -50px; z-index: 0;">
      
    </div>

    <div style="position: relative; z-index: 10;">
      <div id="block-header" style="position: relative;">
        <div style="position: absolute; top: 20px; right: 20px;">
          
        </div>
      </div>

      <table style="width: 100%; border-collapse: collapse;">...</table>
    </div>

  </div>
</body>
</html>

```

4. Laravel Blade Export Strategy (The Translation Layer)

Exporting to Blade requires injecting PHP directives (@foreach, @if, {{ \$var }}) into the HTML. However, React's renderToStaticMarkup will sanitize and ruin raw Blade syntax if we try to render it directly as DOM nodes.

The Solution: The Token Replacement Engine

During the React rendering phase (Pass 1), we use strict, safe string tokens. After the HTML string is generated, we run a Javascript Regex parser (Pass 2) to convert those tokens into valid

Blade directives.

Step A: The JSON Setup

In the builder, the user sets up a dynamic table. The JSON stores this configuration:

JSON

```
{
  "id": "table-1",
  "type": "DynamicTable",
  "props": {
    "repeaterKey": "invoice.items",
    "itemAlias": "item",
    "columns": ["Description", "Price"]
  }
}
```

Step B: The React Render (Pass 1 Output)

The React Exporter generates safe placeholder wrappers:

HTML

```
<tbody>
  <tr>
    <td>[[ item.description ]]</td>
    <td>[[ item.price ]]</td>
  </tr>
</tbody>
```

Step C: The RegEx Transformer (Pass 2 Output)

Before downloading the .blade.php file, the app runs the transformer logic:

1. Convert `` → @foreach(\$invoice->items as \$item)
2. Convert `` → @endforeach
3. Convert [[item.description]] → {{ \$item->description }}

Final Exported Blade File:

Blade

```
<tbody>
  @foreach($invoice->items as $item)
  <tr>
    <td>{{ $item->description }}</td>
    <td>{{ $item->price }}</td>
  </tr>
  @endforeach
</tbody>
```

5. Handling Background Shapes and Patterns

In your provided images, there are complex, patterned header banners (blue/red mesh patterns, diagonal cuts).

- **Builder Implementation:** These should not be built via complex HTML/CSS borders in the export, as PDF engines will break them. Instead, the builder will provide pre-made **SVG Shapes**.
- **Export Implementation:** When exporting, the app will read the user's selected primary color (e.g., #6b21a8 for the purple in your screenshot), inject that hex code directly into the SVG string's <path fill="HEX">, and then convert the entire SVG to a Base64 background image. This guarantees 100% accurate rendering in the final PDF/HTML.

Next Steps for Development

With the export engine defined, we have a clear path for data flow: Builder UI → JSON State → Export Engine → HTML/Blade.

Would you like to move on to designing the **React Component Structure for the Builder UI** (how the sidebar, drag-and-drop context, and toolbars will be laid out), or would you prefer to finalize the **complete JSON schema** for all the specific blocks you need (Invoices, Receipts, Proformas)?